

Import Statement

```
1 import xgboost as xgb
2 import pandas as pd
3 import numpy as np
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score, classification_report
6 from sklearn.preprocessing import LabelEncoder
7 import matplotlib.pyplot as plt
```

Loading the Dataset

```
1 df = pd.read_csv("matches.csv")
2 print("Dataset Shape:", df.shape)
3 print(df.head())
```

Dataset Shape: (1095, 20)

```
   id  season  city  date match_type player_of_match \
0  335982  2007/08  Bangalore  2008-04-18  League  BB McCullum
1  335983  2007/08  Chandigarh  2008-04-19  League  MEK Hussey
2  335984  2007/08  Delhi  2008-04-19  League  MF Maharoor
3  335985  2007/08  Mumbai  2008-04-20  League  MV Boucher
4  335986  2007/08  Kolkata  2008-04-20  League  DJ Hussey

   venue  team1 \
0  M Chinnaswamy Stadium  Royal Challengers Bangalore
1  Punjab Cricket Association Stadium, Mohali  Kings XI Punjab
2  Feroz Shah Kotla  Delhi Daredevils
3  Wankhede Stadium  Mumbai Indians
4  Eden Gardens  Kolkata Knight Riders

   team2  toss_winner  toss_decision \
0  Kolkata Knight Riders  Royal Challengers Bangalore  field
1  Chennai Super Kings  Chennai Super Kings  bat
2  Rajasthan Royals  Rajasthan Royals  bat
3  Royal Challengers Bangalore  Mumbai Indians  bat
4  Deccan Chargers  Deccan Chargers  bat

   winner  result  result_margin  target_runs \
0  Kolkata Knight Riders  runs  140.0  223.0
1  Chennai Super Kings  runs  33.0  241.0
2  Delhi Daredevils  wickets  9.0  130.0
3  Royal Challengers Bangalore  wickets  5.0  166.0
4  Kolkata Knight Riders  wickets  5.0  111.0

   target_overs  super_over  method  umpire1  umpire2
0  20.0  N  NaN  Asad Rauf  RE Koertzen
1  20.0  N  NaN  MR Benson  SL Shastri
2  20.0  N  NaN  Aleem Dar  GA Pratapkumar
3  20.0  N  NaN  SJ Davis  DJ Harper
4  20.0  N  NaN  BF Bowden  K Hariharan
```

Featues and Target

```
1 features = ['team1', 'team2', 'toss_winner', 'toss_decision',
2            'venue', 'city', 'season',
3            'toss_win_match_win'] # new engineered feature
4 target_col = 'winner'
5
6 # Fill missing city with 'Unknown'
7 df['city'] = df['city'].fillna('Unknown')
8
9 # New Feature: Did the toss winner also win the match?
10 # (historical pattern helps the model)
11 df['toss_win_match_win'] = (df['toss_winner'] == df['winner']).astype(int)
12
13 # Drop rows with missing values in selected columns
14 df = df[features + [target_col]].dropna()
15
16 # Target: Did team1 win? → 1 if yes, 0 if no
17 df['target'] = (df['team1'] == df['winner']).astype(int)
18
19 print("\nTarget distribution (1=team1 won, 0=team1 lost):")
20 print(df['target'].value_counts())
```

```
Target distribution (1=team1 won, 0=team1 lost):
target
1    555
```

```
0 535
Name: count, dtype: int64
```

▼ Categorical Columns

```
1 cat_cols = ['team1', 'team2', 'toss_winner', 'toss_decision', 'venue', 'city']
2 le = LabelEncoder()
3 for col in cat_cols:
4     df[col] = le.fit_transform(df[col].astype(str))
5
6 X = df[features]
7 y = df['target']
8
9 print("\nFeatures preview:")
10 print(X.head())
```

```
Features preview:
   team1 team2 toss_winner toss_decision venue city season \
0     16     8          16             1    23     2  2007/08
1      6     0           0             0    40     7  2007/08
2      3    13          13             0    16    10  2007/08
3     10    16          10             0    55    26  2007/08
4      8     1           1             0    14    23  2007/08

   toss_win_match_win
0                   0
1                   1
2                   0
3                   0
4                   0
```

▼ Train/Test Split

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y, test_size=0.2, random_state=42
3 )
```

▼ Train XGBoost Classifier

```
1 baseline_model = xgb.XGBClassifier(
2     n_estimators=5,
3     max_depth=3,
4     learning_rate=1.0,
5     objective='binary:logistic',
6     random_state=42,
7     eval_metric='logloss',
8     # Fix: enable_categorical must be True when using pandas category dtypes
9     enable_categorical=True
10 )
11
12 # Fix: Added 'season' to the list of columns to be converted to 'category'
13 # XGBoost cannot handle 'object' (string) dtypes directly.
14 for col in ['team1', 'team2', 'venue', 'season']:
15     if col in X_train.columns:
16         X_train[col] = X_train[col].astype('category')
17         X_test[col] = X_test[col].astype('category')
18
19 baseline_model.fit(X_train, y_train)
20 y_pred_base = baseline_model.predict(X_test)
21 acc_base = accuracy_score(y_test, y_pred_base)
22 print(f"\nBaseline Accuracy (5 trees, depth=3): {acc_base:.4f}")
```

Baseline Accuracy (5 trees, depth=3): 0.7615

```
1 model = xgb.XGBClassifier(
2     n_estimators=200,          # More trees
3     max_depth=5,              # Deeper trees
4     learning_rate=0.05,       # Slower, more precise learning
5     objective='binary:logistic',
6     subsample=0.8,            # Row sampling per tree
7     colsample_bytree=0.8,     # Feature sampling per tree
8     random_state=42,
9     eval_metric='logloss',
10    # Fix: enable_categorical must be True for the categorical columns in X_train
11    enable_categorical=True
12 )
13
14 model.fit(
15     X_train, y_train,
```

```

16     eval_set=[(X_test, y_test)],
17     verbose=False                # Suppress per-round output
18 )

```

XGBClassifier

```

XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=0.8, device=None, early_stopping_rounds=None,
              enable_categorical=True, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.05, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=5, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=200, n_jobs=None,
              num_parallel_tree=None, ...)

```

✖ Evaluating the model

```

1 y_pred = model.predict(X_test)
2 acc_improved = accuracy_score(y_test, y_pred)
3 print(f"Improved Accuracy (200 trees, depth=5): {acc_improved:.4f}")
4 print(f"\nAccuracy Gain: +{(acc_improved - acc_base)*100:.2f}%")
5
6 # Detailed report
7 print("\nClassification Report:")
8 print(classification_report(y_test, y_pred, target_names=['team2 wins', 'team1 wins']))

```

Improved Accuracy (200 trees, depth=5): 0.8165

Accuracy Gain: +5.50%

Classification Report:

	precision	recall	f1-score	support
team2 wins	0.82	0.82	0.82	110
team1 wins	0.81	0.81	0.81	108
accuracy			0.82	218
macro avg	0.82	0.82	0.82	218
weighted avg	0.82	0.82	0.82	218

✖ Comparison between Baseline and Improved

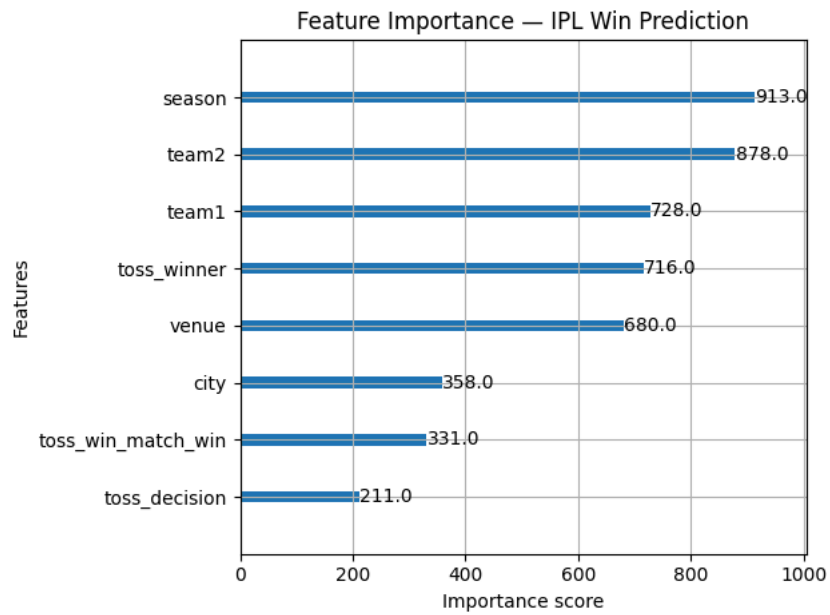
```

1 plt.figure(figsize=(7, 5))
2 bars = plt.bar(
3     ['Baseline\n(5 trees, depth=3)', 'Improved\n(200 trees, depth=5)'],
4     [acc_base * 100, acc_improved * 100],
5     color=['#4C72B0', '#DD8452'],
6     width=0.4
7 )
8 for bar, val in zip(bars, [acc_base, acc_improved]):
9     plt.text(bar.get_x() + bar.get_width()/2,
10             bar.get_height() + 0.3,
11             f"{val*100:.2f}%", ha='center', fontsize=12, fontweight='bold')
12 plt.ylim(50, 100)
13 plt.ylabel("Accuracy (%)")
14 plt.title("Baseline vs Improved XGBoost – IPL Win Prediction")
15 plt.tight_layout()
16 plt.savefig("ipl_accuracy_comparison.png", dpi=150)
17 plt.show()
18 print("Comparison chart saved as ipl_accuracy_comparison.png")

```



<Figure size 800x500 with 0 Axes>



Feature importance saved as inl feature importance.png